

MOBL-99: Best Practice Summary

Series: MOBL — Mobile Monitoring | Notebook: 99 | Created: March 2026 | Last Updated: 03/26/2026

Overview

This notebook consolidates every actionable best practice from the MOBL series (notebooks 01 through 12) into a single, definitive reference. Each best practice specifies the exact setting or value to use, its priority level, and which category it belongs to. Use this as a checklist when instrumenting, configuring, and operating Dynatrace native mobile monitoring.

Table of Contents

- 1. SDK Setup & Configuration
- 2. Crash Reporting & Symbolication
- 3. User Action Instrumentation
- 4. Network Request Monitoring
- 5. Session Replay & Privacy
- 6. Session Properties & User Tagging
- 7. Privacy & Compliance
- 8. Dashboards & Alerting
- 9. SDK Performance Optimization
- 10. Advanced Instrumentation
- 11. DQL Query Patterns

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Grail enabled
Permissions	rum.read , entities.read , bizevents.read , metrics.read
Mobile App	At least one mobile application with Dynatrace SDK integrated
Prior Knowledge	Familiarity with MOBL-01 through MOBL-12 recommended

1. SDK Setup & Configuration

Best practices for installing and configuring the Dynatrace mobile SDK across iOS, Android, and cross-platform frameworks.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Enable auto-start for production apps	iOS: <code>DTXAutoStart = true</code> in Info.plist; Android: <code>autoStart = true</code> in Gradle config	Critical	MOBL-02, MOBL-03
2	Enable crash reporting	iOS: <code>DTXCrashReportingEnabled = true</code> ; Android: <code>crashReporting(true)</code>	Critical	MOBL-02, MOBL-03
3	Use Swift Package Manager over CocoaPods for iOS	Add via Xcode: File > Add Package Dependencies	Recommended	MOBL-02
4	Pin Gradle plugin version to a specific release	<code>id("com.dynatrace.instrumentation") version "8.x.x" -- never use latest</code>	Critical	MOBL-03
5	Use separate Application IDs for iOS and Android	Each platform gets its own Application ID and Beacon URL from Dynatrace	Critical	MOBL-04
6	Use separate Application IDs for prod vs staging	Configure build variant overrides in Gradle; use different Info.plist per scheme	Recommended	MOBL-03
7	Never commit Application IDs or Beacon URLs to public repos	Store in environment variables or <code>local.properties</code> excluded from version control	Critical	MOBL-03
8	Enable hybrid monitoring for WebView apps	iOS: <code>DTXHybridApplication = true</code> ; Android: <code>hybridMonitoring(true)</code>	Recommended	MOBL-02, MOBL-03
9	For Flutter: use <code>dynatrace.config.yaml</code> at project root	Provide platform-specific <code>applicationId</code> and <code>beaconUrl</code> under <code>android:</code> and <code>ios:</code> keys	Critical	MOBL-04
10	For React Native: re-run <code>npx instrumentDynatrace</code> after config changes	Changing <code>dynatrace.config.js</code> does not auto-apply; the instrument command must re-run	Critical	MOBL-04
11	Verify both platform configs exist for cross-platform apps	Query <code>fetch dt.entity.device_application</code> and confirm both Android and iOS entries appear	Critical	MOBL-04

2. Crash Reporting & Symbolication

Best practices for ensuring crash data is complete, readable, and actionable.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Upload dSYM files for every iOS release build	Automate via Xcode Build Phase Script using <code>DTXDssClient</code> , or use Fastlane <code>dynatrace_process_symbols</code>	Critical	MOBL-06
2	Upload ProGuard/R8 mapping files for every Android release build	Enable in Gradle: <code>symbols { apitoken 'YOUR_API_TOKEN' }</code> for automatic upload	Critical	MOBL-06
3	Upload React Native source maps	Set <code>sourceMap: { android: true, ios: true }</code> in <code>dynatrace.config.js</code>	Critical	MOBL-04
4	Verify symbolication status after upload	Check Dynatrace UI under Mobile > Symbol Files; stack traces must show method names, not hex addresses	Critical	MOBL-06
5	For Bitcode iOS builds: download dSYMs from App Store Connect	Local dSYMs do not match Apple-recompiled binaries; download from Xcode Organizer	Critical	MOBL-06
6	Add ProGuard keep rules for Dynatrace SDK	<code>-keep class com.dynatrace.** { *; }</code> and <code>-dontwarn com.dynatrace.**</code> in <code>proguard-rules.pro</code>	Critical	MOBL-03
7	Report handled exceptions via SDK API	iOS: <code>DTXAction.reportError(withName:error:)</code> ; Android: <code>Dynatrace.reportError(name, exception)</code>	Recommended	MOBL-06
8	Use descriptive error names for reported errors	<code>"Payment Failed"</code> not <code>"Error"</code> -- enables meaningful DQL grouping	Recommended	MOBL-06
9	Report errors at catch boundaries, not in utility methods	Attach full context (user action, screen name) where the error is handled	Recommended	MOBL-06
10	Do not report transient/expected conditions as errors	Network retries, expected timeouts, and recoverable conditions create noise	Recommended	MOBL-12
11	Target crash-free rate above 99.5%	< 99%: investigate immediately; 99-99.5%: review top crash groups; > 99.5%: healthy	Critical	MOBL-11

3. User Action Instrumentation

Best practices for tracking user interactions accurately and meaningfully.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Add manual instrumentation for SwiftUI views	Apply <code>.dtAction(name:)</code> to top-level screen views and key interaction buttons	Critical	MOBL-02, MOBL-05
2	Add manual instrumentation for Jetpack Compose	Use <code>Dynatrace.enterAction("name")</code> and <code>action.leaveAction()</code> for all <code>onClick</code> handlers	Critical	MOBL-03, MOBL-05
3	Always call <code>leaveAction()</code> in a <code>finally</code> block	Forgetting to close an action inflates duration metrics and orphans child events	Critical	MOBL-03, MOBL-05
4	Use descriptive, stable action names	"Cart: Add Item" not <code>"btn_click"</code> or <code>"button_42"</code>	Critical	MOBL-05
5	Never put dynamic values in action names	"View Product Details" not "View Product #48291" -- dynamic values cause cardinality explosion	Critical	MOBL-05
6	Never put user IDs or timestamps in action names	Creates infinite unique names, defeats grouping, and is a privacy risk	Critical	MOBL-05
7	Use feature-area prefixes in naming convention	<code>"Cart: Add Item"</code> , <code>"Search: Apply Filter"</code> , <code>"Checkout: Submit Order"</code>	Recommended	MOBL-05
8	Use consistent casing across platforms	<code>"Search Products"</code> on both iOS and Android, not mixed case variants	Recommended	MOBL-05
9	Avoid applying <code>.dtAction()</code> to every SwiftUI subview	Excess actions create noise; apply only to top-level screens and key interactions	Recommended	MOBL-02
10	Configure user action naming rules in Dynatrace UI	Settings > RUM > Mobile > User action naming; test rules against recent data before deploying	Recommended	MOBL-05
11	Keep action hierarchies shallow	Parent + 1 level of child actions maximum for nested actions	Recommended	MOBL-05, MOBL-12

4. Network Request Monitoring

Best practices for HTTP monitoring, backend correlation, and network performance optimization.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Instrument backend services with OneAgent or OpenTelemetry	Required for <code>x-dtc</code> header correlation; enables end-to-end distributed tracing from device to database	Critical	MOBL-07
2	Use manual web request tagging for non-standard transports	Add <code>x-dynatrace</code> header via <code>DTXAction.getRequestTag()</code> (iOS) or <code>Dynatrace.getRequestTag()</code> (Android) for WebSocket, gRPC, or custom HTTP clients	Recommended	MOBL-12
3	Enable gzip/brotli compression on API responses	Reduces response payload size by 60-80%	Recommended	MOBL-07
4	Implement response pagination	Avoid loading entire datasets in a single request	Recommended	MOBL-07
5	Use HTTP/2 or HTTP/3 for multiplexing	Reduces connection overhead and round trips	Recommended	MOBL-07
6	Implement client-side caching with ETags and Cache-Control	Prevents redundant network requests for unchanged resources	Recommended	MOBL-07
7	Serve static assets from a CDN	Reduces DNS, TCP, and TLS times for geographically distributed users	Recommended	MOBL-07
8	Adapt app behavior to connection type	Reduce image quality, defer non-critical requests, enable offline queueing on slow connections	Optional	MOBL-07
9	When TTFB is the dominant phase, investigate backend	Use frontend-to-backend correlation to trace into server-side services and database calls	Recommended	MOBL-07
10	Exclude analytics/CDN URLs from SDK monitoring	Configure <code>excludeURLs</code> patterns for <code>analytics.google.com</code> , <code>firebaseanalytics.googleapis.com</code> , CDN domains	Recommended	MOBL-12

5. Session Replay & Privacy

Best practices for configuring mobile session replay with appropriate privacy controls.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Enable session replay on crash	iOS: <code>DTXSessionReplayOnCrash = true</code> ; Android: <code>sessionReplayOnCrash(true)</code>	Critical	MOBL-08
2	Set privacy masking to Safe as the default	iOS: <code>DTXSessionReplayPrivacyMode = SAFE</code> ; Android: <code>sessionReplayPrivacyMode("SAFE")</code>	Critical	MOBL-08
3	Set production sample rate to 5-10%	iOS: <code>DTXSessionReplaySampleRate = 10</code> ; Android: <code>sessionReplaySampleRate(10)</code>	Critical	MOBL-08
4	Set staging/QA sample rate to 100%	Full coverage for pre-release testing and bug verification	Recommended	MOBL-08
5	Use Safest masking for financial/healthcare apps	Replays show layout and navigation but no readable content	Critical	MOBL-08
6	Only use Custom masking when you need to unmask specific elements	Start with Safe, move to Custom only for targeted debugging; tag specific views with <code>dtxMaskingMode</code>	Recommended	MOBL-08
7	Temporarily increase sample rate during incidents	Raise to 50-100% when actively investigating a reported issue; lower again after resolution	Recommended	MOBL-08
8	Monitor DEM unit consumption for session replay	Track in Dynatrace license overview; session replay is the primary DEM cost driver	Recommended	MOBL-08
9	Consider shorter Grail retention for session replay data	Session replay contains more sensitive visual information than raw telemetry	Optional	MOBL-09

6. Session Properties & User Tagging

Best practices for enriching sessions with business context and identifying users.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Tag users with opaque IDs, never PII	<code>Dynatrace.identifyUser("usr_a1b2c3d4")</code> -- never email addresses, phone numbers, or full names	Critical	MOBL-09
2	Clear user tag on logout	iOS: <code>Dynatrace.identifyUser(nil)</code> ; Android: <code>Dynatrace.identifyUser(null)</code>	Critical	MOBL-09
3	Set session properties early in the session	Call <code>reportValue()</code> immediately after login or app start so properties apply to all subsequent actions	Recommended	MOBL-09

#	Best Practice	Recommended Setting/Value	Priority	Source
4	Limit to 20-30 custom session properties per app	Excessive properties increase beacon size and processing overhead	Recommended	MOBL-09
5	Use enum-like string values for properties	<code>"tier" = "free" not "tier" = "Free Trial Account"</code> -- enables clean aggregation	Recommended	MOBL-09
6	Use consistent property key names across iOS and Android	Same keys on both platforms so a single DQL query covers both	Recommended	MOBL-09
7	Never store PII in session property values	Session properties are not subject to data masking	Critical	MOBL-09
8	Report A/B test variants as session properties	<code>reportValue("ab_test_checkout_v2", "variant_b")</code> -- enables correlation with performance/crash data	Recommended	MOBL-12

7. Privacy & Compliance

Best practices for GDPR, CCPA, and general data privacy compliance.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Implement opt-in mode for GDPR compliance	Set <code>DTXAutoStart = false</code> (iOS) or <code>autoStart = false</code> (Android); start SDK only after user consent	Critical	MOBL-09
2	Default data collection level to Performance until consent	<code>DataCollectionLevel.PERFORMANCE</code> captures anonymous performance data without user identification	Critical	MOBL-09
3	Persist user consent in app storage	SDK does not persist consent; store in <code>UserDefaults</code> (iOS) or <code>SharedPreferences</code> (Android) and check on each launch	Critical	MOBL-09
4	Provide a way to withdraw consent in app settings	Set data collection level to <code>Off</code> when user revokes consent	Critical	MOBL-09
5	Implement data deletion API for right-to-erasure requests	POST to <code>/api/v2/data-privacy/deletion</code> with <code>userId</code> , <code>startDate</code> , <code>endDate</code> , <code>dataTypes: ["RUM"]</code>	Critical	MOBL-09

#	Best Practice	Recommended Setting/Value	Priority	Source
6	Set shortest Grail retention period that meets business needs	Configure dedicated Grail buckets with retention matching your data minimization policy	Recommended	MOBL-09
7	Separate crash reporting opt-in from general monitoring	Crash reporting has its own <code>crashReportingOptedIn</code> flag independent of data collection level	Recommended	MOBL-09
8	Display clear, plain-language consent dialog	Explain what data is collected, why, and how; provide granular options for Performance vs User Behavior levels	Critical	MOBL-09
9	Consider separate Grail buckets for EU vs non-EU data	Use OpenPipeline rules to route data based on geolocation for data residency compliance	Optional	MOBL-09

8. Dashboards & Alerting

Best practices for operationalizing mobile monitoring with dashboards and alerts.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Include a crash-free rate single-value tile with color thresholds	Green: > 99.5%, Yellow: 98-99.5%, Red: < 98%	Critical	MOBL-11
2	Organize dashboard: health at top, trends in middle, drill-down at bottom	Top: single-value tiles (crash rate, sessions); Middle: timeseries (trends); Bottom: tables (top crashes, slowest actions)	Recommended	MOBL-11
3	Add a dashboard variable for <code>useraction.application</code>	Lets stakeholders filter to their specific app	Recommended	MOBL-11
4	Set default time range to 24 hours with presets for 1h, 6h, 24h, 7d	Covers most operational use cases without excessive data scanning	Recommended	MOBL-11
5	Alert on high crash rate	Static threshold: > 10 crashes in a 1-hour sliding window, severity Critical	Critical	MOBL-11
6	Alert on session volume drop	< 50% of rolling 7-day baseline, severity Warning	Recommended	MOBL-11
7	Alert on slow app launch	Average app start > 5 seconds over a 15-minute window, severity Warning	Recommended	MOBL-11

#	Best Practice	Recommended Setting/Value	Priority	Source
8	Alert on high HTTP error rate	> 5% of mobile requests returning 5xx, severity Critical	Critical	MOBL-11
9	Use Dynatrace Intelligence for adaptive baselines on performance metrics	Dynatrace Intelligence automatically learns patterns; add static thresholds only for hard SLA limits	Recommended	MOBL-11
10	Tag mobile app entities with <code>app-type: mobile</code>	Enables detected problem workflow triggers filtered to mobile-specific problems	Recommended	MOBL-11
11	Create audience-specific dashboards	Developers: 5-min refresh, crash detail; QA: 15-min, crash-free rate by version; Executives: daily KPI snapshot	Recommended	MOBL-11
12	Break down crash rate by <code>app.version</code> after every release	Identifies if a specific release introduced a regression	Critical	MOBL-10, MOBL-11

9. SDK Performance Optimization

Best practices for minimizing SDK overhead on battery, network, and CPU.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Exclude third-party analytics and CDN URLs from monitoring	Add <code>excludeURLs</code> patterns: <code>analytics.google.com/*</code> , <code>firebaseanalytics.googleapis.com/*</code> , CDN domains	Recommended	MOBL-12
2	Use sampling for high-traffic production apps	10-50% sampling reduces DEM consumption proportionally; even 1% of millions provides thousands of sessions	Recommended	MOBL-08, MOBL-12
3	Start with full monitoring, reduce only if needed	Do not prematurely optimize; measure SDK overhead first	Recommended	MOBL-12
4	Use crash-only mode for ultra-low overhead scenarios	Set performance collection level to crash-only; captures crashes and errors with minimal overhead	Optional	MOBL-12
5	App launch time target: under 2 seconds	< 1s: excellent; 1-2s: acceptable; 2-5s: needs improvement; > 5s: investigate immediately	Critical	MOBL-10, MOBL-11

10. Advanced Instrumentation

Best practices for custom business events, A/B testing, multi-app strategies, and advanced SDK usage.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Use reverse-domain naming for custom business event types	<code>"com.myapp.purchase"</code> not <code>"purchase"</code> - - avoids collisions across apps and services	Recommended	MOBL-12
2	Send business events for conversion-critical interactions	Purchases, sign-ups, feature usage via <code>sendBizEvent()</code> with structured attributes	Recommended	MOBL-12
3	Report feature flag evaluations as business events	<code>event.type = "com.myapp.feature_flag"</code> with <code>feature.name</code> and <code>feature.variant</code> attributes	Optional	MOBL-12
4	Use nested actions for multi-step flows	Parent action wraps the full flow; child actions measure individual steps (validate, submit, confirm)	Optional	MOBL-12
5	For long-running actions, manage lifecycle explicitly	Call <code>leaveAction()</code> in completion handler or <code>finally</code> block; do not rely on auto-close timeout	Critical	MOBL-12
6	Use consistent attribute names across iOS and Android business events	Same keys on both platforms enable unified DQL queries	Recommended	MOBL-12
7	Never include PII in business event attributes	No email addresses, phone numbers, or personal data in custom event payloads	Critical	MOBL-12
8	Use consistent naming prefixes for multi-app organizations	<code>"MyBrand Customer"</code> , <code>"MyBrand Driver"</code> , <code>"MyBrand Admin"</code> for easy DQL filtering	Recommended	MOBL-12
9	Budget DEM units per app based on criticality	100% monitoring for critical apps; use sampling for high-traffic secondary apps	Recommended	MOBL-12
10	Attach <code>reportValue()</code> properties to actions for business context	<code>action.reportValue(withName: "total_price", doubleValue: 49.99)</code> -- becomes queryable in Grail	Recommended	MOBL-12

11. DQL Query Patterns

Mandatory patterns and filters for querying mobile data in Grail.

#	Best Practice	Recommended Setting/Value	Priority	Source
1	Always filter mobile data on event.provider	<code>filter event.provider == "www.dynatrace.com/mobile"</code> - - this is the key discriminator for mobile events	Critical	MOBL-10
2	Use <code>countDistinct(dt.rum.session.id)</code> for unique session counts	Never use <code>count()</code> for sessions -- each session generates many events	Critical	MOBL-10
3	Use conditional countDistinct for crash rate	<code>countDistinct(if(event.type == "com.dynatrace.crash", then:dt.rum.session.id, else:null))</code>	Recommended	MOBL-10
4	Add <code>isNotNull()</code> filters before grouping on optional fields	Fields like <code>os.type</code> , <code>device.model</code> , <code>app.version</code> may be null; filter first to avoid noise	Recommended	MOBL-10
5	Query mobile app entities for inventory	<code>fetch dt.entity.device_application</code> -- no time range needed; returns current state	Recommended	MOBL-10
6	Filter crashes with <code>event.type == "com.dynatrace.crash"</code>	Reported errors use <code>event.type == "com.dynatrace.error.report"</code> -- distinguish between the two	Critical	MOBL-06, MOBL-10
7	Always specify explicit time ranges on bizevents queries	<code>fetch bizevents, from:-1h</code> or <code>from:-24h</code> -- never rely on the default 2-hour window	Critical	MOBL-10
8	Segment by <code>app.version</code> for release monitoring	Enables crash rate and performance comparison across releases	Recommended	MOBL-10

Priority Summary

Priority	Count	Description
Critical	36	Must implement -- failure to do so causes data loss, security risk, broken monitoring, or compliance violation
Recommended	39	Should implement -- significantly improves data quality, operational efficiency, or user experience
Optional	5	Nice to have -- advanced capabilities for mature mobile monitoring programs

Implementation Order

1. **First:** All Critical items in SDK Setup, Crash Reporting, and Privacy sections
 2. **Second:** Critical items in User Action Instrumentation and Dashboards/Alerting
 3. **Third:** All Recommended items
 4. **Last:** Optional items for advanced use cases
-

References

- [Dynatrace Mobile Monitoring Documentation](#)
 - [iOS SDK Integration Guide](#)
 - [Android SDK Integration Guide](#)
 - [Session Replay for Mobile](#)
 - [Mobile SDK Privacy Settings](#)
-

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.